



East West University

RESEARCH PAPER

Project (Milestone-3)

Semester: Summer 2026

Course Title: Artificial Intelligence

Course Code: CSE366

Sec: 05

**nl2sql-366: A Lightweight Hybrid Machine-Learning and Rule-Based System
for**

Natural Language to SQL Query Generation

Repository: <https://github.com/alfahim-fuyad/nl2sql-366>

Demonstration: <https://nl2sql-366.onrender.com/>

Group Number: 02

Submitted by-

Md Al Fahim Fuyad

ID: 2023-1-60-066

Md Toyabur Rhaman

ID: 2023-1-60-065

Fahmida Mahin

ID: 2023-3-60-481

Submitted to-

Md Al-Imran

Senior Lecturer

Department of Computer Science & Engineering

East West University

Date: 20/08/2026

nl2sql-366: A Lightweight Hybrid Machine-Learning and Rule-Based System for Natural-Language SQL Query Generation

End-to-End Workflow of the nl2sql-366 System

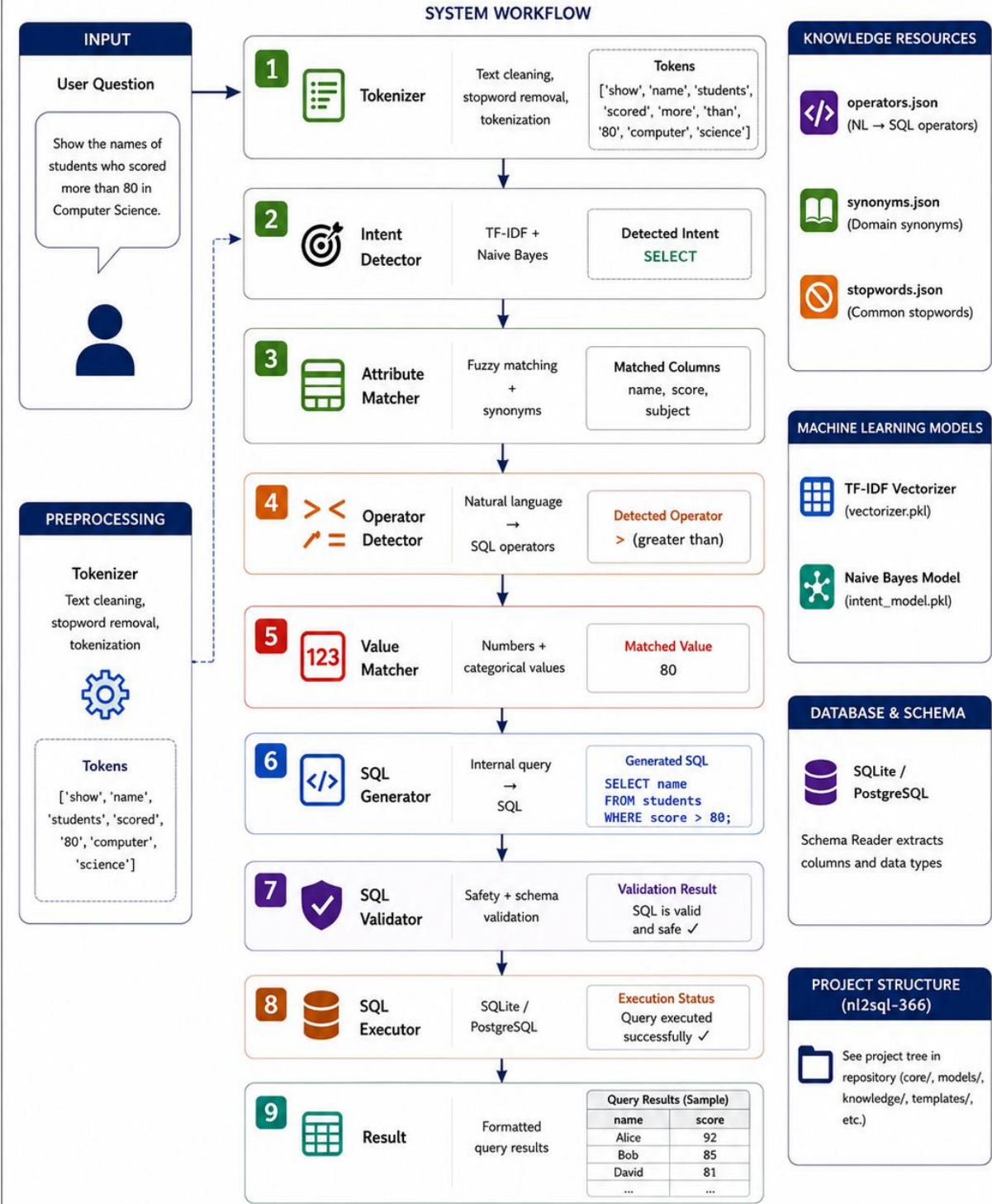


Figure 1: End-to-End Workflow of the nl2sql-366 System

```

nl2sql-366/
├── app.py                # Flask web server
├── main.py              # Interactive CLI
├── core/
│   ├── attribute_matcher.py # Fuzzy column name matching
│   ├── dataset_loader.py   # CSV → SQLite / PostgreSQL
│   ├── intent_detector.py  # ML intent classifier
│   ├── operator_detector.py # NL phrase → SQL operator
│   ├── response.py         # CLI result formatter
│   ├── schema_reader.py   # DataFrame schema extraction
│   ├── sql_executor.py    # Query execution
│   ├── sql_generator.py   # Internal query → SQL string
│   ├── sql_validator.py   # Safety + correctness checks
│   ├── tokenizer.py       # Text cleaning utilities
│   └── value_matcher.py   # Number + categorical extraction
├── knowledge/
│   ├── operators.json     # NL phrase → SQL operator mapping
│   ├── stopwords.json    # Words to ignore during matching
│   └── synonyms.json     # Domain synonym dictionary (300+ entries)
├── models/
│   ├── intent_model.pkl   # Trained Naive Bayes classifier
│   ├── vectorizer.pkl     # Fitted TF-IDF vectorizer
│   └── train_intent.py    # Training script
├── training_data/
│   ├── intent_dataset.csv # 100,000 labelled training examples
│   ├── generate_dataset.py # Synthetic dataset generator
│   └── convert_wikisql.py # WikiSQL → intent format converter
├── tests/
│   ├── conftest.py
│   ├── test_dataset_and_schema.py
│   ├── test_matchers.py
│   ├── test_sql_generation_and_execution.py
│   ├── test_tokenizer_and_operator.py
│   └── test_validator.py
├── templates/
│   ├── index.html        # Main web UI template (composes the partials below)
│   └── partials/
│       ├── navbar.html   # Top navbar + mobile hamburger menu
│       ├── about_modal.html # "About" modal content
│       ├── help_modal.html # "Help" modal content
│       ├── upload_card.html # Drag & drop upload card
│       ├── preview_card.html # Dataset preview table
│       └── query_card.html # Question input + SQL/results display
├── favicon.svg
├── static/
│   ├── css/
│   │   ├── base.css      # Reset, body, container, header, card
│   │   ├── navbar.css    # Navbar + mobile hamburger menu
│   │   ├── modal.css     # Modal shell + About modal styling
│   │   ├── upload.css    # Drop zone, upload strip, data requirements
│   │   └── query-results.css # Ask row, SQL output, results table, spinner
│   ├── images/
│   │   └── ewu-logo.png
│   └── js/
│       ├── dom.js        # Shared DOM element references
│       ├── utils.js      # Shared UI helpers (errors, escaping)
│       ├── upload.js     # Drag & drop / file upload logic
│       ├── query.js      # Ask question + render results
│       ├── navbar.js     # Logo reload + mobile hamburger menu
│       └── modals.js     # About/Help modal open & close behavior
├── data/
│   └── sample.csv        # Sample dataset for testing
├── simulation.html       # Interactive pipeline visualiser (learning tool)
├── requirements.txt      # List of required Python packages
├── Procfile             # Starts the Flask app on Render
└── render.yaml          # Render deployment settings

```

Natural Language to SQL

Upload a CSV or Excel dataset and ask questions in plain English — the AI generates and runs the SQL for you.

● dataset.csv

1,000 rows · 19 columns

[Change file](#)

DATA REQUIREMENTS

- First row must contain column headers.
- Supports CSV (.csv) and Excel (.xlsx, .xls) files with text and numeric columns.
- Descriptive column names improve matching accuracy.
- Large datasets work fine — only schema and sample values are held in memory.

DATASET PREVIEW

Showing first 5 of 1,000 rows

Gender	Age	NS1	IgG	IgM	Area	AreaType	HouseType	District	Fever_Duration	Body
Female	45	0	0	0	Mirpur	Undeveloped	Building	Dhaka	2	36.9
Male	17	0	0	1	Chawkbazar	Developed	Building	Dhaka	1	36.5
Female	29	0	0	0	Paltan	Undeveloped	Other	Dhaka	1	36.6
Female	63	1	1	0	Motijheel	Developed	Other	Dhaka	7	38.9
Male	22	0	0	0	Gendaria	Undeveloped	Building	Dhaka	1	37

Ask a question about your data

How many female patient are there?

Ask

GENERATED SQL

Intent: COUNT

```
SELECT COUNT(*) FROM "data" WHERE "Gender" = 'Female'
```

QUERY RESULTS

1 row returned.

count

524